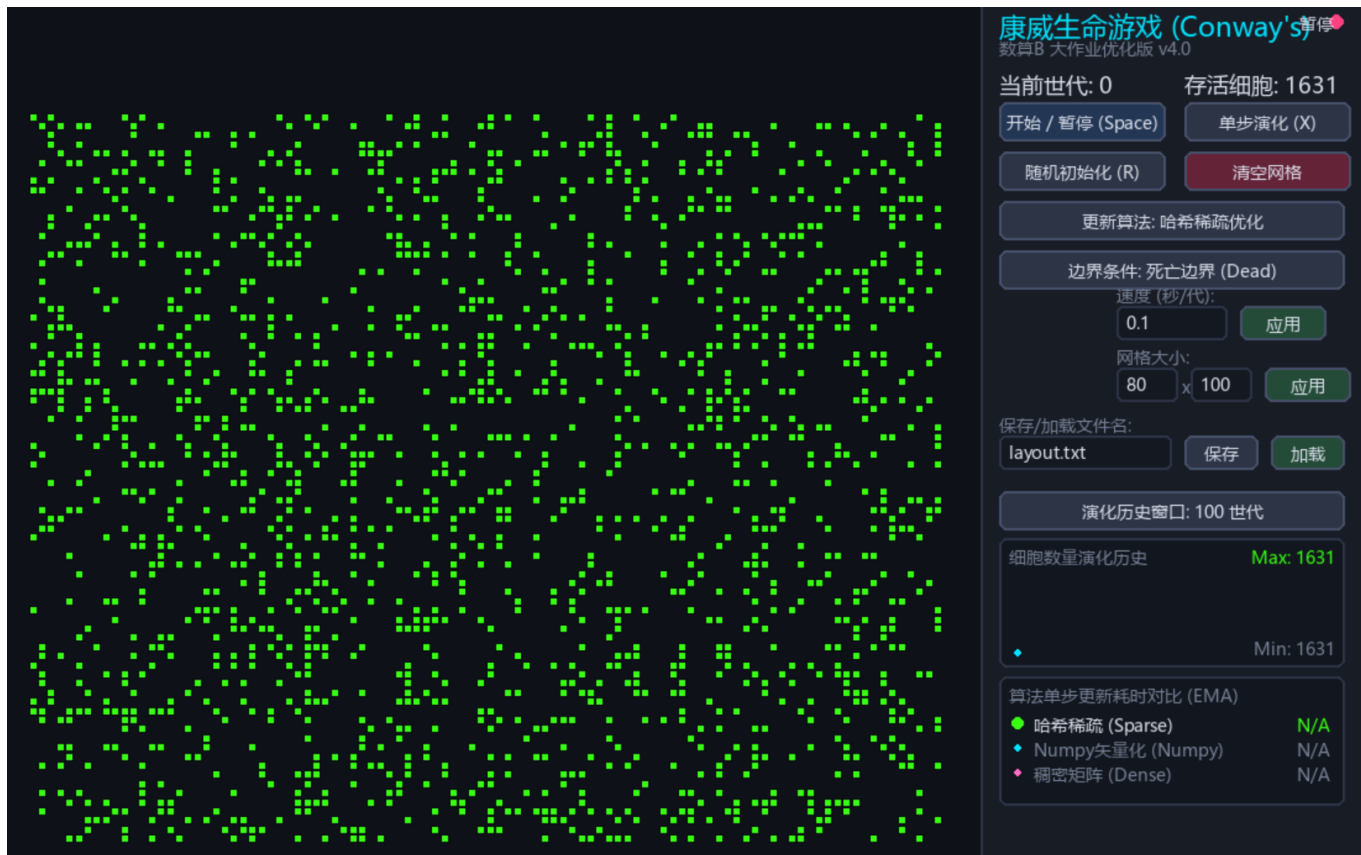


康威生命游戏 (Conway's Game of Life) - 算法优化与性能分析版

仓库链接: [GitHub - zh-rm/Game_of_Life: Conway's Game of Life · GitHub](#)

实现并应用了哈希稀疏表示算法, 并在 GUI 中集成了三种不同算法的实时耗时 (ms) 对比面板和细胞存活数量实时演化折线图。



一、项目亮点与选题初衷

经典的康威生命游戏通常采用二维稠密矩阵存储, 并在每代更新时通过双重循环扫描整个网格 ($O(W \times H)$ 复杂度)。当网格规模很大且细胞分布稀疏 (例如 1000×1000 的稀疏宇宙) 时, 这种算法会产生海量的无效计算和内存消耗。

为了将课程所学的计算机基础知识深度应用于实际问题, 本项目:

- 深度应用数据结构:** 实现基于哈希表 (Python `set`) 的**稀疏哈希算法 (Sparse Hash-Set)**, 仅存储活跃细胞, 将更新时间复杂度优化到 $O(A)$, 其中 A 为活跃/存活细胞数, 消除了大网格下的冗余计算。

2. **实时性能可视化评测：** 在 GUI 中设计了**算法更新耗时对比面板** (EMA 指数滑动平均处理)，直观展现了哈希稀疏优化在不同密度下的效率提升。
3. **精美交互界面：** 采用现代科技暗黑风界面 (Pygame 驱动)，支持鼠标滚轮在光标处缩放、中/右键平移拖拽、以及左键拖拽连续“绘制/擦除”细胞，带有实时的细胞演化折线图。

二、核心算法设计与复杂度分析

本项目在 `src/game_logic.py` 中独立实现了以下三种状态更新算法：

1. 哈希稀疏网格算法 (Sparse Hash-Set) — [深度课程应用]

- **核心思想：** 下一代可能改变状态的细胞，必须满足：**其本身是存活细胞，或它是某个存活细胞的邻居**。因此，所有非存活细胞且不与任何存活细胞相邻的细胞在下一代中必然保持死亡状态。
- **具体实现：**
 1. 使用 Python 哈希表 `self.sparse_grid = set()` 存储所有存活细胞的坐标 `(row, col)`。
 2. 创建一个临时计数哈希字典 `neighbor_counts = {}`。
 3. 遍历 `self.sparse_grid` 中的每一个存活细胞，将其周围 8 个邻居的计数值加 1。这一步会自动处理边界（如 `loop` 循环边界或 `dead` 死亡边界）。
 4. 遍历整个 `neighbor_counts`（仅包含有存活邻居的细胞）：
 - 如果一个细胞的存活邻居数等于 3，则它在下一代变为存活。
 - 如果等于 2 且原本就在 `self.sparse_grid` 中，它继续存活。
 - 其余情况判定为死亡。
- **复杂度分析：**
 - **时间复杂度：** 仅为 $O(A)$ ，其中 A 为当前存活细胞的个数。在稀疏状态下，它不仅远优于 $O(W \times H)$ 的普通稠密矩阵，在极大规模网格上甚至领先于 NumPy 矢量化实现。
 - **空间复杂度：** $O(A)$ 。只需要存储当前存活的细胞及其邻居的计数信息。

2. 稠密矩阵算法 (Dense Loop) — [基准对照组]

- **具体实现：** 使用二维列表 `self.dense_grid = [[0]*cols for _ in range(rows)]`，双重循环逐个扫描每一个细胞，在每一代通过双缓冲区创建新网格，再进行指针交换。
- **复杂度分析：**
 - **时间复杂度：** 恒定为 $O(W \times H)$ 。不管存活细胞数量多寡，每次都要全部扫描。
 - **空间复杂度：** $O(W \times H)$ ，必须维护完整的二维网络。

3. Numpy 矢量化算法 (Numpy Vectorized) — [硬件加速组]

- **具体实现：** 利用 Numpy 的二维矩阵整体移位 `np.roll`（用于循环边界）或填充切片 `np.pad`（用于死亡边界）进行八方向求和，在 C 语言层面进行并行处理，从而避免 Python 本身循环的慢速。
- **复杂度分析：**
 - **时间复杂度：** $O(W \times H)$ ，但常数项极小。
 - **空间复杂度：** $O(W \times H)$ 。

4. 算法效率随细胞密度变化的对比分析

细胞密度 / 网格状态	稠密矩阵算法 (Dense Loop)	Numpy 矢量化算 法 (Numpy)	哈希稀疏算法 (Sparse Hash-Set)
大规模网格 + 极稀疏 (e.g. Glider)	极慢（需要计算 10^6 次循环）	较快（底层的矩阵 平移）	极快（仅计算十余次 哈希操作）
中等规模网格 + 随机 均匀密度	慢	最快（发挥硬件级 加速）	较快
全满/极稠密矩阵	最慢	最快	较慢（哈希冲突与常 数开销）

三、工程与文件结构

本项目遵循标准的 GitHub 开源规范进行工程化组织：

```
├─ src/                # 核心源代码
│  ├─ main.py          # 主程序入口，整合事件逻辑与渲染循环
│  ├─ game_logic.py    # 生命游戏三大更新算法及数据转换
│  ├─ gui.py           # UI组件 (Button, TextInput, Selector, Chart, Stats)
│  ├─ hashlife_runner.py # 百万世代超高速 CLI 模拟器 (基于 Hashlife 算法)
│  └─ hashlife_gui.py   # Hashlife 模拟器的 Pygame 图形化前端 (4 页交互流程)
├─ tests/              # 测试用例
│  └─ test_logic.py    # 自带运行器的单元测试 (验证算法一致性与边界)
├─ requirements.txt    # 环境依赖清单
├─ LICENSE             # MIT 开源许可协议
└─ README.md           # 本项目主说明文档
```

四、界面与交互指南

运行程序后，您将看到一个深色调的精美界面：

1. 左侧游戏区域：

- **平移滚动：** 鼠标**中键**或**右键**长按并拖拽，可以平移画布视口。
- **缩放：** 鼠标滚轮向上（放大）、向下（缩小），缩放将**严格以鼠标指针为中心**。
- **手动编辑细胞：** 暂停状态下，在游戏区域**单击或长按拖拽**。长按拖拽支持智能“涂鸦绘制”或“橡皮擦除”（取决于拖拽起始格的状态），且在单次拖拽中不会重复反转同一个格点。

2. 右侧控制面板：

- **基础控制：** 开始/暂停 (Space)、单步演化 (X)、随机初始化 (R)、清空格点 (C)。
- **参数应用：** 可实时调节速度 (s/gen, 即每代秒数) 和网格大小（修改行和列后点击应用会自动重置画布并居中）。
- **布局保存与加载：** 在输入框内输入文件名（如 `layout.txt`），点击保存即可导出细胞坐标；清屏后点击加载可完美复原。
- **实时图表与 Benchmarking：** 实时绘制细胞数量演化曲线，支持通过选择器切换历史窗口大小（50代、100代、200代、500代，或全量无限模式）；统计面板实时更新三大算法单步执行的时间（毫秒级）。

五、运行与环境配置

1. 环境依赖安装

本项目支持 Python 3.8+。推荐安装 `numpy` 和 `pygame`。

```
pip install -r requirements.txt
```

(注：即使您的环境未安装 `numpy`，程序也会自动退回到 `Sparse / Dense` 算法运行，不会发生崩溃。)

2. 启动游戏

在项目根目录下执行：

```
python src/main.py
```

3. 运行自动化测试

运行在 `tests` 下编写的针对算法演化一致性、边界 wrap-around 及 File I/O 的测试套件：

```
python tests/test_logic.py
```

4. 运行百万世代超高速模拟 (Hashlife CLI)

如果您想在一秒内运行完 1,000,000 世代的演化并给出演化结果（利用 1G 内存空间缓存空间拓扑实现时空跳转），请执行：

```
python src/hashlife_runner.py -G 1000000 -W 500 -H 500
```

参数说明：

- `-W / --width`：网格边界宽度（默认 500）
- `-H / --height`：网格边界高度（默认 500）
- `-G / --generations`：模拟的代数（默认 1000000 代）
- `-I / --input`：输入初始格点文件路径（默认读取 `layout.txt`，若无则自动使用 `--density` 随机初始化）
- `-O / --output`：输出结果格点文件路径（默认保存至 `output_result.txt`）
- `--density`：初始随机细胞密度（默认 0.15）

六、学术诚信与 AI 工具声明

本项目核心的哈希稀疏状态更新算法参考了相关研究。使用了 AI 辅助进行 Pygame UI 组件的部分样板代码编写、界面配色的微调润色、文本的润色以及测试用例的部分生成。

七、许可证

本项目基于 [GPL-3.0 license](#) 开源。